

COURSE CODE/TITLE: CPE 010-CPE21S1 - Data Structures and Algorithms

SECTION: CPE21S1

DATE PERFORMED: Sep 1, 2026

DATE SUBMITTED: Sep 1, 2026

NAME: Mendoza, Roy Gabriel I.

INSTRUCTOR: Engr. Jimlord Quejado

ACTIVITY NO.7.2

Seatwork 7.2: Algorithm Analysis: Best-Case and Worst-Case Time Complexity

1. PROCEDURE OUTPUT

1. Finding an Element

```
C/C++  
for (int i = 0; i < n; i++)  
{  
    if (arr[i] == target)  
    {  
        return i;  
    }  
}
```

>> Best-Case Time Complexity

>> $O(1)$

Worst-Case Time Complexity

>> $O(n)$

Brief explanation of how you arrived at your answers

>> the loop searches in sequence for target, if the target is at first, then it is just 1, but if it is at the last, then it could be at n or the limit of the array.

2. Printing All Elements

```
C/C++  
for (int i = 0; i < n; i++)  
{  
    cout << arr[i] << endl;  
}
```

>> Best-Case Time Complexity

>> $O(n)$

Worst-Case Time Complexity

>> $O(n)$

Brief explanation of how you arrived at your answers

>> the loop is executed as many times as the contents of the array(until the end of the loop which is unknown n)

3. Nested Loops

```
C/C++  
for (int i = 0; i < n; i++)  
{  
    for (int j = 0; j < n; j++)  
    {  
        cout << i << " " << j << endl;  
    }  
}
```

>> **Best-Case Time Complexity**

>> $O(n^2)$

Worst-Case Time Complexity

>> $O(n^2)$

Brief explanation of how you arrived at your answers

>> Both the inner and outer loop run the entire range of the array.

4. Conditional Processing

```
C/C++  
if (n > 100)  
{  
    for (int i = 0; i < n; i++)  
    {  
        cout << arr[i];  
    }  
}  
else  
{  
    cout << "Small input";  
}
```

>> **Best-Case Time Complexity**

>> $O(1)$

Worst-Case Time Complexity

>> $O(n)$

Brief explanation of how you arrived at your answers

>> If the first given is already has the value of >100 then the loop can end there, but if it is <100, then the loop will run/iterate until it finds something that is >100.

5. Fixed Inner Loop

```
C/C++  
for (int i = 0; i < n; i++)  
{  
    for (int j = 0; j < 10; j++)  
    {  
        cout << i << j;  
    }  
}
```

>> **Best-Case Time Complexity**

>> **O(n)**

Worst-Case Time Complexity

>> **O(n)**

Brief explanation of how you arrived at your answers

>> Since the inner loop is limited to 10, we can just make it O(10) but since the outer loop has not end It is an O(n) without any stop.

6. Two Consecutive Loops

```
C/C++  
for (int i = 0; i < n; i++)  
{  
    cout << arr[i];  
}  
  
for (int j = 0; j < n; j++)  
{  
    cout << arr[j] * 2;  
}
```

>> **Best-Case Time Complexity**

>> **O(n)**

Worst-Case Time Complexity

>> **O(n)**

Brief explanation of how you arrived at your answers

>> The 2 independent loops have no end and are running unconditionally.

7. Dividing by Two

```
C/C++  
int x = n;
```

```
while (x > 1)
{
    x = x / 2;
}
```

>> **Best-Case Time Complexity**

>> $O(\log n)$

Worst-Case Time Complexity

>> $O(\log n)$

Brief explanation of how you arrived at your answers

>> Each iteration halves the value of x until it is no longer than 1, and since it is a division, it is automatically log of n.

8. Nested Loop With Different Limits

```
C/C++
for (int i = 0; i < n; i++)
{
    for (int j = 0; j < i; j++)
    {
        cout << i << j;
    }
}
```

>> **Best-Case Time Complexity**

>> $O(n^2)$

Worst-Case Time Complexity

>> $O(n^2)$

Brief explanation of how you arrived at your answers

>> Unlike in example 5, this is a nonstop inside of a nonstop, basically $x * x = x^2$.

9. Early Termination

```
C/C++
for (int i = 0; i < n; i++)
{
    if (arr[i] == target)
    {
        cout << "Found";
        break;
    }
}
```

>> **Best-Case Time Complexity**

>> O(1)

Worst-Case Time Complexity

>> O(n)

Brief explanation of how you arrived at your answers

>> If the target is at index 0, the loop can already stop, but if it is at the last, it will run for an n amount of iteration until it is found.

10. Two-Level Search

```
C/C++
for (int i = 0; i < n; i++)
{
    for (int j = 0; j < n; j++)
    {
        if (arr[i] == target)
        {
            break;
        }
    }
}
```

>> **Best-Case Time Complexity**

>> O(n)

Worst-Case Time Complexity

>> O(n²)

Brief explanation of how you arrived at your answers

>> The inner loop will run until all the data have been traversed, then if the data has been found for like the "array 2" of 1, then it could stop, but if it is at the last, then it will run it all until it is found causing the x*x instead of the x+1 like the best case.

11. Linear Loop With Logarithmic Inner Work

```
C/C++
for (int i = 0; i < n; i++)
{
    int x = n;

    while (x > 1)
    {
        x = x / 2;
    }
}
```

>> **Best-Case Time Complexity**

>> O(n log n)

Worst-Case Time Complexity

>> **$O(n \log n)$**

Brief explanation of how you arrived at your answers

>> the inner loop performs the $\log n$ iteration like in example 7, then the outer performs just an n so $n \cdot \log n = n \log n$.

12. Conditional Nested Loop

```
C/C++
if (arr[0] == target)
{
    cout << "Found";
}
else
{
    for (int i = 0; i < n; i++)
    {
        for (int j = 0; j < n; j++)
        {
            cout << arr[i] + arr[j];
        }
    }
}
```

>> **Best-Case Time Complexity**

>> **$O(1)$**

Worst-Case Time Complexity

>> **$O(n^2)$**

Brief explanation of how you arrived at your answers

>> The best case shows that if the `arr[0]` is already the target, we can iterate it just 1 time, but if not it would perform a loop inside of a loop for an n amount of time inside of a loop for n amount of time.

13. Finding the Maximum

```
C/C++
int max = arr[0];

for (int i = 1; i < n; i++)
{
    if (arr[i] > max)
    {
        max = arr[i];
    }
}
```

>> **Best-Case Time Complexity**

>> **$O(n)$**

Worst-Case Time Complexity

>> $O(n)$

Brief explanation of how you arrived at your answers

>> Since the iteration will run to be compared against the current max, the iteration will run for n in both the best and worst.

14. Three Nested Loops

```
C/C++
for (int i = 0; i < n; i++)
{
    for (int j = 0; j < n; j++)
    {
        for (int k = 0; k < n; k++)
        {
            cout << i + j + k;
        }
    }
}
```

>> **Best-Case Time Complexity**

>> $O(n^3)$

Worst-Case Time Complexity

>> $O(n^3)$

Brief explanation of how you arrived at your answers

>> The 3 loops are nested and there are no break, this will be practically be a $n*n*n$.

15. Comprehensive Analysis

```
C/C++
for (int i = 0; i < n; i++){
    if (arr[i] == target){
        return i;}
    int x = n;
    while (x>1){
        x = x/2;}}
```

>> **Best-Case Time Complexity**

>> $O(1)$

Worst-Case Time Complexity

>> $O(n \log n)$

Brief explanation of how you arrived at your answers

>> the loop searches in sequence for target, if the target is at first, then it is just 1, but if it is at the last, then it could be at n or the limit of the array.

If the arr[0] is already the target, we can run it just 1 time, but if it is not, the while function inside of the for loop will be used for an n amount of time causing an $n \cdot \log n$.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Copy and paste the questions from the lab and include your answers after.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: None

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: None

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: None

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.



Please have the honor of pledge in the comment section or at the bottom of your activity document/exam or any form of file submission.

I affirm that I will not give or receive any unauthorized help on this activity/exam and that all work will be my own.